



Impact of AI-agent-supported collaborative learning on the learning outcomes of University programming courses

Haoming Wang¹ · Chengliang Wang¹ · Zhan Chen¹ · Fa Liu¹ · Chunjia Bao² · Xianlong Xu¹ 

Received: 25 October 2024 / Accepted: 4 March 2025 / Published online: 13 March 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

With the rapid development of artificial intelligence technology in the field of education, AI-Agents have shown tremendous potential in collaborative learning. However, traditional Computer-Supported Collaborative Learning (CSCL) methods still have limitations in addressing the unique demands of programming education. This study proposes an innovative AI-Agent-supported Collaborative Learning (AI-CL) approach, aimed at enhancing students' programming learning experience and effectiveness through intelligent assistance from AI-Agents. The research developed an AI-Agent system based on large language models and employed a quasi-experimental design to investigate the effects of the AI-CL method on students' learning achievement, self-efficacy, cognitive load, and learning interest. The experiment was conducted at a university in Shanghai, recruiting 45 undergraduate students who were randomly assigned to the AI-CL experimental group ($n=24$) and the CSCL control group ($n=21$). The experimental context was set in an ACM programming competition teaching environment, lasting for 6 weeks with 70-min sessions each week. Results showed that the AI-CL group significantly outperformed the CSCL group in learning achievement, self-efficacy, and learning interest. Regarding cognitive load, the AI-CL group demonstrated significantly lower mental effort compared to the CSCL group, while there was no significant difference in mental load. These findings not only provide new theoretical perspectives for the application of cognitive load theory and self-efficacy theory in AI-assisted learning environments but also offer strong practical guidance for higher education institutions to introduce AI-Agent-assisted collaborative learning models in programming courses.

Keywords AI-Agent · Large language models · Collaborative learning · Programming education · Learning outcomes

Haoming Wang and Chengliang Wang contributed equally to this manuscript.

Extended author information available on the last page of the article

1 Introduction

In today's digital age, programming ability has become one of the most critical skills of the twenty-first century, having significant impact on both individual career development and socio-economic progress (Jing et al., 2024a; Kalelioğlu, 2015; Wang et al., 2025a). As a result, higher education institutions are increasingly incorporating programming courses into their curriculum across various disciplines (Salleh et al., 2010). However, the complexity and abstractness of programming learning bring enormous challenges to many students, leading to issues such as learning anxiety, low self-efficacy, and cognitive overload (Ghanizadeh, 2017; Makransky & Mayer, 2022). These difficulties not only affect students' learning achievement but may also decrease their interest in and long-term participation in learning (Lee, 2014). Therefore, exploring effective teaching strategies to improve the learning achievement of programming courses has become an urgent topic in current higher education research.

Computer-Supported Collaborative Learning (CSCL), as an innovative teaching model, has been widely applied in programming education in recent years. CSCL promotes interaction and collaboration among students, contributing to joint knowledge construction and deep learning (Jeong et al., 2019). However, traditional CSCL modes have limitations in addressing special needs in programming learning. For instance, students may feel confused due to the lack of immediate feedback or learning achievement could be affected due to differences in collaborators' abilities (Belland et al., 2017). Additionally, in complex programming tasks, students may face the risk of cognitive overload, thereby reducing the effectiveness of collaborative learning (Wang & Hwang, 2017).

The rapid development of Artificial Intelligence (AI) provides new possibilities for resolving these challenges (Chen et al., 2024; Lin & Yu, 2024; Wang et al., 2025b). AI-Agent-supported Collaborative Learning (AI-CL) has become a highly promising research direction. AI-Agent, with their ability to provide immediate feedback, personalized guidance, and adaptive support, may help overcome the limitations of traditional CSCL (Cress & Kimmerle, 2023). For example, Allcoat & von Mühlenen (2018) found that using VR technology in traditional teaching leads to a better memory level and significantly improves learners' overall learning achievement and emotional engagement. Likewise, an adaptive learning platform based on a fuzzy expert system developed by Hwang et al. (2020) effectively helped underperforming students alleviate learning anxiety and successfully complete their learning tasks.

However, despite the potential shown by AI tools in fields such as language learning, their application effects in programming education remain insufficiently studied (Zawacki-Richter et al., 2019). The uniqueness of programming learning—such as the importance of logical thinking, the complexity of error debugging, and the difficulty of understanding abstract concepts—may require more specialized AI-Agent design (Chen et al., 2020). Besides, existing research mainly focuses on cognitive outcomes, while studies on non-cognitive factors (such as self-efficacy and learning interest) are relatively few (Smiderle et al.,

2020). Considering these factors are vitally important for long-term success in programming learning (Armoni et al., 2015), it becomes particularly necessary to comprehensively evaluate the multi-dimensional impact of AI-CL on programming learning.

Given the aforementioned context, this study aims to systematically explore the impact of AI-CL on university students' learning achievement in programming courses. Specifically, we will evaluate the effects of AI-CL from both cognitive and non-cognitive dimensions. The research questions proposed in this study are as follows:

RQ1: Are the solutions created by the AI-CL experimental group superior to the CSCL control group?

RQ2: Does the AI-CL experimental group require less cognitive load compared to the CSCL control group?

RQ3: Does the AI-CL experimental group exhibit higher self-efficacy than the CSCL control group?

RQ4: Does the AI-CL experimental group exhibit higher learning interest in solving programming tasks compared to the CSCL control group?

2 Literature review

2.1 Development status of CSCL and innovative applications of AI

Computer-Supported Collaborative Learning (CSCL) is an educational method that utilizes computer technology to support and enhance group learning (Järvelä & Hadwin, 2013). CSCL emphasizes interaction and knowledge co-construction among learners, aiming to improve learning achievement and foster collaboration skills (Lin et al., 2023a, 2023b; Rienties et al., 2020). The application of CSCL in programming education has shown positive results. Research by Estácio et al. (2015) demonstrated that pair programming can enhance students' problem-solving abilities and code quality. Nawahdah and Taji's (2016) long-term study further confirmed that students who participated in pair programming were more likely to complete their Computer Science courses and perform better in subsequent courses. However, the application of CSCL in programming education also faces challenges. Capdeferro and Romero (2012) warned that without proper guidance and structure, CSCL might lead to frustration and unequal participation among students. Hernández-Sellés et al. (2019) pointed out that the lack of social presence in online CSCL environments could impact the quality of collaboration. To address these challenges, Chi and Wylie (2014) proposed using scripts to structure CSCL activities, while Leeuwen et al. (2019) developed an AI-based CSCL feedback system to help teachers identify and resolve collaboration issues in a timely manner.

In recent years, the development of AI technology has brought new opportunities to CSCL (Wang et al., 2024a). Hwang et al. (2020) noted that AI can serve as the knowledge provider, discussion facilitator, team coordinator, or learning evaluator. These roles are enabled by an AI system's capability to analyze

interaction patterns amongst learners, thus providing timely and effective interventions to foster deeper collaboration (Jin et al., 2025a; Ouyang et al., 2023). At the application level, Bansal et al. (2024) explored the potential of AI-driven discussion assistants in enhancing student engagement and discussion quality. In programming education, Kimmel et al. (2024) revealed the significant impact of AI-supported programming learning in improving student learning efficiency and programming skills. The system they developed can analyze students' code in real-time, providing personalized error diagnosis and improvement suggestions. This innovation not only greatly reduces students' frustration but also significantly decreases teachers' workload, thus optimizing the overall teaching process. Specifically, Laak and Aru (2024) studied intelligent tutoring systems based on large language models (LLM). They found that such systems could dynamically adjust teaching content and difficulty based on students' learning progress, offering more targeted support for students at different learning levels. These studies indicate that AI technology is driving CSCL from a traditional supportive tool to an intelligent learning partner.

2.2 Cognitive load theory in technology-enhanced learning

Cognitive Load Theory provides a critical theoretical perspective for evaluating the effectiveness of technology-enhanced learning environments. Based on the study by Hwang et al. (2013), cognitive load can be precisely divided into two dimensions: mental load (objective task demands) and mental effort (subjective exertion intensity). In CSCL environments, cognitive load exhibits unique distribution patterns. Empirical studies indicate that learners require substantial cognitive resources for socio-cognitive processes. For instance, in collaborative programming, cognitive load significantly increases due to coordination complexity (Buchner et al., 2022). Research has found that learners' cognitive resources are primarily consumed by communication management (e.g., conflict resolution, role negotiation) and workflow synchronization (Janssen & Kirschner, 2020; Kirschner et al., 2018).

The rise of AI has introduced new possibilities for cognitive load regulation. Intelligent tutoring systems have demonstrated significant effectiveness in reducing intrinsic load (Ouyang et al., 2023). For example, decision-guiding chatbot for interdisciplinary learning that enhances learning efficiency and alleviates task-induced mental load through real-time feedback (Iku-Silan et al., 2023). Studies on AI-supported systems have found that users experience significantly lower subjective cognitive stress in image recognition tasks, as assessed through physiological indicators such as pupil dilation tests (Hudon et al., 2021). AI-assisted learners are able to shift from high cognitive load task decomposition to low cognitive load solution validation, thereby systematically optimizing cognitive resource allocation (Ackerman, 2019; Hoch et al., 2023). However, the differential impact of AI on mental load and mental effort has not yet been fully explored, as most studies remain constrained within the traditional cognitive load theory framework proposed by Sweller (1994).

2.3 Influencing factors of self-efficacy in collaborative settings

Self-efficacy, as a core concept of social cognitive theory, is reflected in programming education as learners' confidence in completing programming tasks (Bandura & Wessels, 1997; Tsai, 2019). Traditional research suggests that learners' self-efficacy in programming can be enhanced in various ways, including observing peer experiences (Fang et al., 2022), receiving timely feedback (Sun & Hsu, 2019), and receiving emotional support (Lin et al., 2023a, b). However, Yildiz Durak (2018) points out that without a properly designed learning framework, learners may experience frustration and disengagement. This is particularly evident in programming learning contexts, as learners often face the dual pressures of understanding abstract concepts and debugging errors in their programs.

With the integration of AI technology into collaborative learning environments, the factors influencing self-efficacy have undergone new changes (Jing et al., 2024b). Maples et al. (2023) found that many learners view AI agents as "mirrors" for social and cognitive interaction, and this unique human–machine relationship enables learners to gain more confidence through interaction with AI. AI systems based on LLM can accurately assess learners' abilities and dynamically adjust task difficulty to maintain the optimal challenge zone, thus creating continuous success experiences. For example, the collaborative programming platform developed by Fogli et al. (2022), based on natural language dialogue, lowers the technical threshold, allowing learners to efficiently complete programming tasks. Thanks to its human-like interaction, this type of intelligent collaborative environment also effectively addresses the lack of social presence in online collaboration (Chen et al., 2025; Dai et al., 2024; Hernández-Sellés et al., 2019; Jin et al., 2025b; Wang et al., 2023). However, despite the great potential of AI in education, as Zhang et al. (2024) cautioned, over-reliance on AI assistance may reduce learners' willingness to independently explore complex problems.

2.4 Learning interest in programming education

Learning interest, as the learner's persistent attention and positive emotional experience toward the learning content, is a key factor influencing the effectiveness of programming education (Renninger & Hidi, 2011). Due to the high abstraction and practical characteristics of programming learning, students are prone to frustration caused by difficulties in understanding, which in turn leads to a decline in learning interest (Lindberg et al., 2019). Research by Higashi et al. (2017) suggests that during the learning process of core abilities like program abstraction thinking and algorithm design, maintaining learning interest directly affects students' sustained engagement. Traditional teacher-centered teaching models struggle to provide personalized scaffolding, which causes learners to lose interest in the early stages due to the lack of immediate feedback (Benítez et al., 2024). As a result, researchers have started exploring CSCL methods and introducing gamified teaching mechanisms to stimulate situational interest. Liu et al. (2022) found that using Scratch for

programming learning can significantly enhance learners' interest in learning. However, subsequent analysis revealed that the improvement in learning outcomes was positively correlated with learning interest, and this correlation was particularly significant for learners with initially low interest. This suggests that traditional methods still have limitations in stimulating learning interest, prompting researchers to explore more effective teaching support strategies.

Current research frontiers focus on the synergistic role of AI in stimulating learning interest. AI primarily reconstructs the mechanisms of interest development through three major pathways: First, task difficulty adjustment based on learner ability can optimize the learning experience and sustain learning interest. For example, the intelligent programming assistant developed by Pirzado et al. (2024) analyzes code quality in real time and breaks tasks into adaptive micro-goals, ensuring that the task difficulty stays within the Zone of Proximal Development (ZPD). Second, humanized interaction design enhances learners' learning interest. Wang et al. (2024b) found that AI-driven personalized assistants, through intelligent problem-generation strategies, not only improve learners' understanding but also stimulate sustained learning interest. Third, AI-supported collaborative learning paradigms further expand the dimensions of cultivating learning interest. Empirical research by Su et al. (2024) showed that intelligent collaborative learning environments with high-quality system services could enhance learners' satisfaction and learning interest, thus promoting continuous learning behavior. This intelligent collaborative mechanism analyzes learners' interaction patterns in real time, optimizes group strategies and discussion guidance, ensuring that the collaborative process always stays within the optimal interest-stimulating range (Baucal et al., 2023).

3 AI-agent-supported collaborative programming approach in a CSCL environment

This study introduces an innovative method for collaborative programming, allowing learners to utilize an AI-Agent for intelligent assistance in a CSCL environment and enabling interaction with the AI-Agent and team members at any time. This method draws upon the Collaborative Problem-Solving model proposed by Graesser et al. (2018) and the Pair Programming teaching strategy suggested by Wei et al. (2021) as an effective way to organize collaborative programming activities. Figure 1 illustrates the implementation of the AI-CL method, which includes two primary components: the conventional collaboration part and the AI-Agent part. These two parts are tightly integrated via task allocation and a feedback mechanism, forming a complete collaborative programming ecosystem. The conventional collaboration part adheres to the classic collaborative programming process involving five key stages: task analysis, pseudocode writing, coding, debugging, and optimization. These stages form a cyclical iterative process that allows participants to continuously improve their codes based on task objectives. Throughout this process, learners can share knowledge and solve issues through direct interaction.

The AI-Agent part constitutes the core innovation of this method. It introduces AI-Agents, which is based on LLM, to serve as an intelligent collaborative partner.

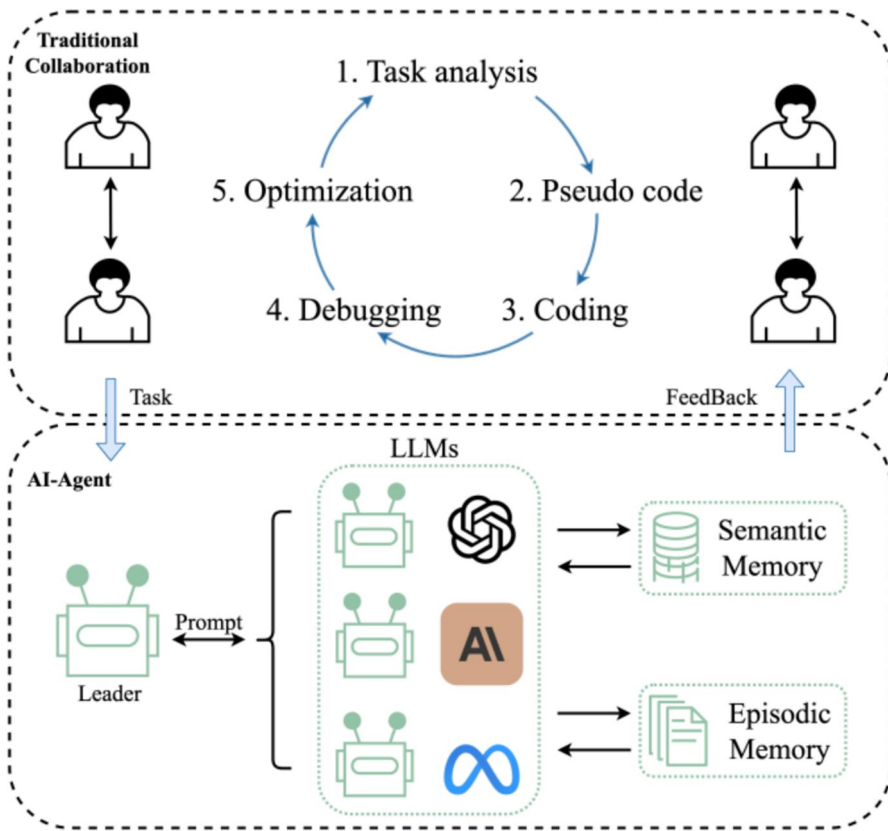


Fig. 1 AI-agent-supported collaborative programming approach

The AI-Agent interacts with Prompts and multiple LLM, selectively invoking different LLM according to task requirements to fully leverage the strengths of each model. Furthermore, the system includes two crucial memory components: Semantic Memory and Episodic Memory. Semantic Memory stores and manages concepts, rules, and knowledge related to programming, enabling the AI-Agent to understand the programming context and provide relevant suggestions. The Episodic Memory records historical information and solutions of specific programming tasks, helping the AI-Agent provide more targeted assistance in similar situations. The unique feature of this architecture is its ability to dynamically adapt to various programming tasks and learner needs. For example, during the code-generation phase, the system might prefer to use an LLM excelling at code generation; while explaining concepts, it might select a model more proficient in natural language processing. Simultaneously, with the memory system, the AI-Agent can accumulate experience over time, providing increasingly accurate assistance.

In practical applications, learners first carry out task analysis, goal setting, and role allocation in the traditional collaboration environment. Subsequently, team

members begin the actual collaborative programming work. During this process, the AI-Agent acts as an intelligent collaborative partner, providing continuous support. When learners encounter difficulties, they can seek assistance from the AI-Agent via a specially designed interface (as shown in Fig. 2). The AI-Agent can flexibly invoke the most suitable model according to different task needs and, in combination with information from its Semantic Memory and Episodic Memory, generate relevant suggestions. Throughout the entire collaboration process, the method continuously monitors tasks and assesses codes. This includes automatic evaluation by the AI-Agent, peer evaluation within the group, and professional evaluation by teachers. The AI-Agent not only provides technical support in this process but also enhances collaboration effectiveness in the CSCL environment by facilitating communication, interaction, and evaluation. It creates an intelligent and adaptive learning environment, effectively promoting the comprehensive development of learners' programming skills, problem-solving abilities, and team collaboration skills. Through this approach, the AI-CL method effectively integrates AI technology with collaborative learning theory, providing an innovative teaching model in the field of computer education, and significantly improving the effectiveness and learning experience of collaborative programming in the CSCL environment.

4 Experimental design

4.1 Participants and experimental environment

This study aims to investigate the impact of AI-CL methods on improving the programming learning achievement of university students. The research team conducted a quasi-experimental study at a university in Shanghai to evaluate the effectiveness of AI-Agent application in programming education. A total of 45 undergraduate students were recruited as participants. All students had completed at least one programming language course and related computer algorithm courses, and eligibility was confirmed through a screening questionnaire.

After ensuring all participants had a thorough understanding of the study's purpose and procedures and signed informed consent forms, the research adopted a

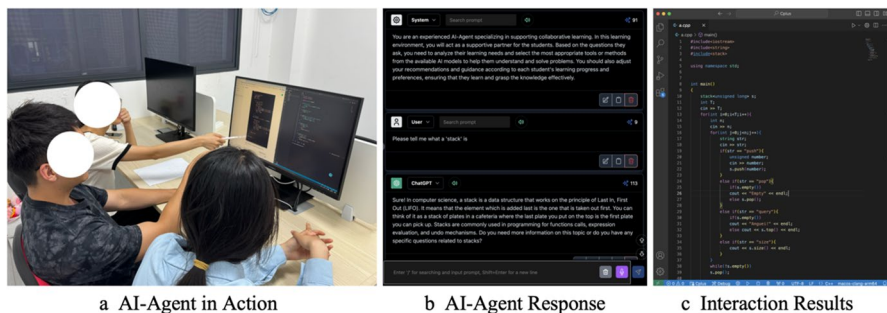


Fig. 2 Application scenarios of AI-agent in the experiment

randomized controlled design for group assignment (Mo et al., 2022; Zhang et al., 2025). Each participant was given a unique identification number, and a random grouping tool developed using Python's Random library was used for random assignment. To ensure the objectivity of the experiment, the grouping process was independently conducted by a research assistant who was not involved in the subsequent outcome evaluation. Ultimately, the participants were divided into two groups: the AI-CL group ($n=24$), which used AI-Agent-supported collaborative learning for programming, and the CS-CL group ($n=21$), which used traditional computer-supported collaborative learning methods. Two groups of students were organized into study teams, each consisting of three members. The AI-CL group formed 8 teams, while the CS-CL group formed 7 teams. Each team member assumed different roles during collaboration, including algorithm design, code implementation, and debugging/verification. These roles were rotated throughout the tasks to ensure that every learner could gain a comprehensive learning experience.

The experiment is set in the teaching environment of the Association for Computing Machinery International Collegiate Programming Contest (ACM-ICPC) to evaluate the effectiveness of the AI-Agent in real-world programming education scenarios. ACM-ICPC is a globally recognized computer algorithm competition, where participating teams (teams of three) are required to solve complex algorithmic problems, with a focus on data structure application, algorithm optimization, and teamwork skills. The teaching environment follows the typical training model of the ACM competition, which is centered around Problem-Based Learning (PBL). The experiment is conducted in the computer laboratory of a university, using past ACM-ICPC regional contest problems as the primary learning resources. The AI-CL group will additionally receive customized learning materials and instant feedback provided by the AI-Agent to explore its impact on the programming learning process and learning achievement.

4.2 Experimental procedure

The experimental procedure is shown in Fig. 3. The entire experiment lasted 6 weeks, with 70 min of experimental time each week. In the first week, we conducted pre-tests on students' learning achievement, self-efficacy, and learning interest, providing baseline data for subsequent analysis. In the second week, students learned the basics of data structures and programming to prepare for the subsequent experiments, ensuring all subjects had the necessary prerequisite knowledge. In the third week, we familiarized students with the collaborative learning process and cultivated their team cohesion through activities. This phase aimed to lay the foundation for subsequent collaborative programming tasks, ensuring students could effectively work in teams. In the fourth week, the experimental group students received training on the learning model with AI-Agent intervention, while the control group became familiar with the learning model in the traditional programming environment. This grouped training ensured that both groups of students were adequately prepared in their respective learning environments. Starting from the fifth week, both groups of students formally entered the

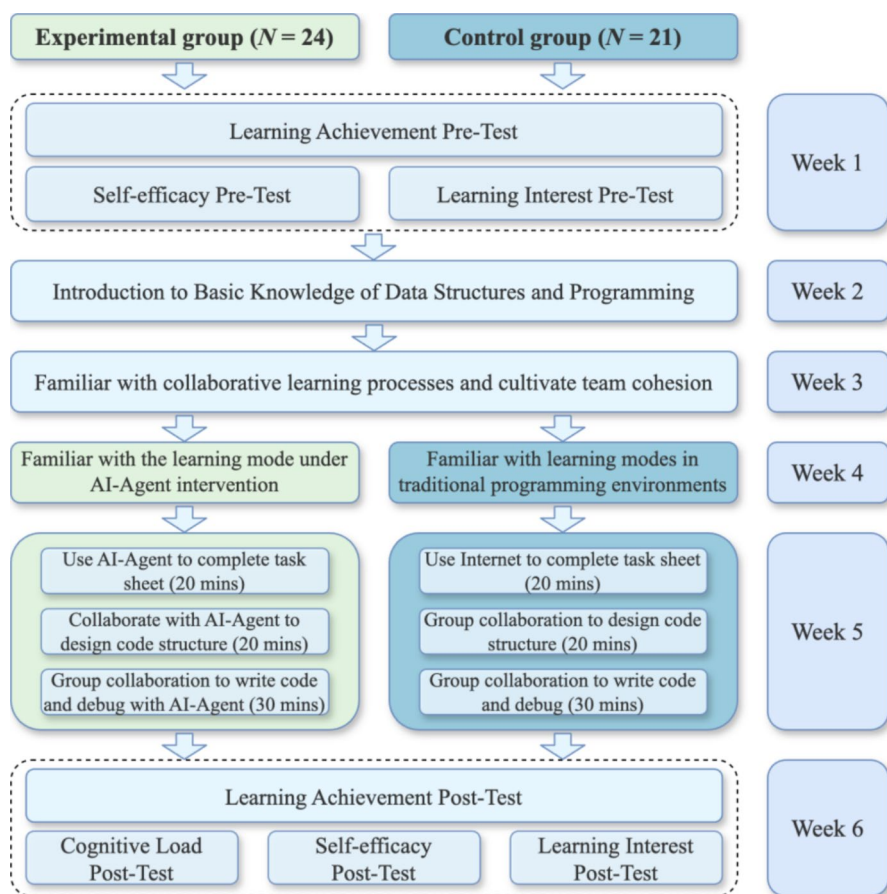


Fig. 3 Experimental design flowchart

experimental phase, completing collaborative programming-based learning activities. In the sixth week, we conducted post-tests on students' cognitive load, self-efficacy, and learning interest. Simultaneously, teachers evaluated students' work as a post-test indicator of learning achievement, comprehensively assessing the impact of AI-Agent intervention on students' programming learning.

Prior to the experiment, the research team designed multi-level difficulty questions covering the programming concept of 'stack'. Through a preliminary experiment, 5 subjects with relevant programming backgrounds were recruited for testing. The system recorded and analyzed the subjects' completion times and performance. The pre-experiment results showed that the questions were of moderate difficulty, with subjects taking an average of 50 min to complete all tasks with AI-Agent assistance. Based on these data, the research team optimized the experimental materials and established recruitment criteria for formal subjects.

Before the formal experiment, we conducted a basic information survey of the subjects, collecting data on their programming backgrounds to ensure an even distribution of subjects with different programming ability levels. Meanwhile, the research team standardized the experimental environment, providing all subjects with identical programming environments and tools to eliminate potential differences arising from the use of varied programming tools. Furthermore, to further eliminate potential interference from tool operation on the experimental results, we also provided subjects with detailed instructions on tool usage, ensuring all participants could skillfully operate these tools.

At the beginning of the experiment, participants were first given a task sheet (see Appendix E for details), guiding the experimental group to use the AI-Agent to learn relevant knowledge about "stacks" through dialogue, while the control group conducted self-directed learning on the internet for approximately 20 min. According to the specific requirements in the task sheet, the experimental group used the AI-Agent's code architecture suggestions and solutions to collaboratively design the code structure, while the control group used an online whiteboard tool for traditional collaborative design. This phase lasted about 20 min. Finally, during the programming phase, participants were required to complete code writing and debugging within 30 min. The AI-CL group's AI-Agent was able to provide targeted solutions and collaborate with group members to optimize the code, whereas the CS-CL group relied entirely on intra-group collaboration and discussion to solve programming problems.

Finally, during the evaluation phase of the experiment, a double-blind review mechanism was employed. Two professional instructors, who were not involved in the experiment, independently evaluated the students' programming projects based on pre-established scoring criteria. Throughout the experiment, the supervising instructor adhered strictly to a neutral support strategy, monitoring the progress of each group in real time through the laboratory's surveillance system. The instructor only intervened in cases of significant obstacles that could impede task completion. Even then, intervention was standardized to indirect prompts (e.g., 'Please revisit the third requirement in the task sheet') to avoid providing direct solutions and ensure the validity of the experiment.

4.3 Instrument

The tools used in this research include tests and questionnaires concerning learning achievement, cognitive load, self-efficacy, and learning interest. The pre-test and post-test of academic performance were jointly developed by two experienced computer science teachers, referencing the basic programming concepts testing framework (Tsai et al., 2024). These tests aimed to assess students' foundational knowledge and their final learning achievement (see Appendix A for details). The pre-test consisted of 20 multiple-choice questions (5 points each), with a total of 100 points. The overall Cronbach's alpha was 0.72. The post-test included four observation points—the completion status of the code, code completeness, quality of data structure construction, and code readability—with each observation point comprising

4–6 scoring items. Every scoring item was worth up to 5 points, adding up to a total of 100 points. The overall Cronbach's alpha was 0.80.

The cognitive load questionnaire was modified from the measure developed by Hwang et al. (2013). The questionnaire used a 5-point Likert-type rating scheme (1 = strongly disagree; 5 = strongly agree) and consisted of eight items (Appendix B). The overall Cronbach's alpha was 0.88. An example item of this measure is: "The content of this programming task is very difficult for me."

The self-efficacy questionnaire was modified from the measure developed by Rowbotham and Schmitz (2013). The questionnaire used a 5-point Likert-type rating scheme (1 = strongly disagree; 5 = strongly agree) and consisted of four items (Appendix C). The overall Cronbach's alpha was 0.93. An example item of this measure is: "I believe I can successfully complete the programming task."

The learning interest questionnaire was modified from the measure developed by Mazer (2013). The questionnaire used a 5-point Likert-type rating scheme (1 = strongly disagree; 5 = strongly agree) and consisted of five items (Appendix D). The overall Cronbach's alpha was 0.94. An example item of this measure is: "The topics covered by this programming task are exciting to me."

5 Result

5.1 Learning achievement

The study employed one-way ANCOVA to assess the learning achievement of two groups of students. The independent variable was the collaborative learning model, the dependent variable was the post-test score of learning achievement, and the covariate was the pre-test score of learning achievement. The Levene's test was not significant ($F(1, 43) = 0.27, p > 0.05$), indicating no violation of the homogeneity of variance assumption. The test of homogeneity of regression slopes was also non-significant ($F(1, 41) < 0.01, p > 0.05$), justifying the use of ANCOVA. As shown in Table 1, the adjusted means for the experimental group and the control group were 83.10 and 78.60, respectively. There was a significant difference in learning achievement between the two types of collaborative learning models ($F(1, 42) = 10.21, p = 0.003 < 0.05$), with a medium effect size ($\eta^2 = 0.196$). The results indicate that, compared to students using CSCL, students learning with AI-CL not only achieved significantly higher learning achievement but also submitted work of noticeably higher quality (see Appendix F for details). These findings further confirm that AI-CL can effectively enhance student learning achievement.

Table 1 The one-way ANCOVA result of learning achievement

Group	N	Mean	SD	Adjust M	Adjust SD	F	η^2
Experimental group	24	83.50	6.39	83.10	0.95	10.21**	0.196
Control group	21	78.14	5.54	78.60	1.02		

** $p < 0.01$

Table 2 Independent-sample T-test result of the cognitive load on the post-questionnaire scores of the two groups

Group	N	Mean	SD	t	Cohen's d
Experimental group	24	2.57	0.68	2.10*	0.63
Control group	21	2.98	0.61		

* $p < 0.05$ **Table 3** Independent-sample T-test result of the cognitive load dimensions on the post-questionnaire scores of the two groups

	Group	N	Mean	SD	t	Cohen's d
Mental Effort	Experimental group	24	2.55	0.65	3.19**	0.95
	Control group	21	3.15	0.60		
Mental Load	Experimental group	24	2.59	0.76	1.32	
	Control group	21	2.88	0.71		

** $p < 0.01$

5.2 Cognitive load

The study also employed an independent samples t-test to compare the cognitive load between the experimental and control groups. As shown in the Table 2, the mean and standard deviation of cognitive load scores for the experimental group were 2.57 and 0.68, respectively, while those for the control group were 2.98 and 0.61. The results of the independent samples t-test indicated a significant difference in cognitive load scores between the two groups ($t = 2.10$, $p = 0.043 < 0.05$), suggesting that the AI-Agent collaborative learning model can reduce students' cognitive load during collaborative programming.

The study further compared two aspects of cognitive load: mental effort and mental load, as shown in Table 3. For the mental effort dimension, the mean and standard deviation for the experimental group were 2.55 and 0.65, respectively, while those for the control group were 3.15 and 0.60. Results from the independent samples t-test indicated that the control group had significantly higher mental effort than the experimental group ($t = 3.19$, $p = 0.003 < 0.05$). On the other hand, for mental load, the mean and standard deviation for the experimental group were 2.59 and 0.76, respectively, while those for the control group were 2.88 and 0.71. The independent samples t-test results showed no significant difference in mental load scores between the two groups ($t = 1.32$, > 0.05). To more clearly present the differences in cognitive load, mental effort, and mental load between the two groups, we created a visualization (Fig. 4).

5.3 Self-efficacy

One-way ANCOVA was used to assess students' self-efficacy. The independent variable was the collaborative learning model, the dependent variable was the post-test score of self-efficacy, and the covariate was the pre-test score. The

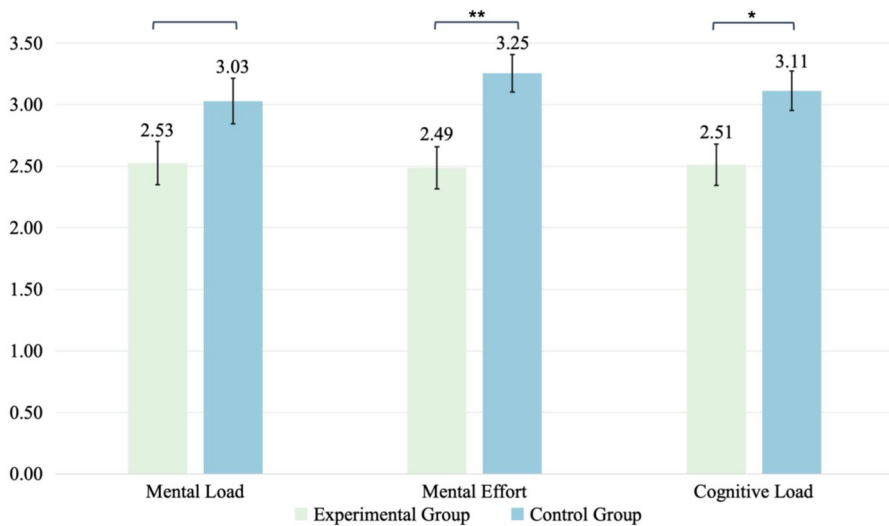


Fig. 4 Post-test numerical graph of different dimensions of cognitive load (* $p < 0.05$, ** $p < 0.01$)

Levene's test was not significant ($F(1, 43) = 1.96, p > 0.05$), indicating no violation of the homogeneity of variance assumption. The results also passed the test for homogeneity of regression slopes ($F(1, 41) = 2.70, p > 0.05$). As shown in Table 4, the adjusted means for the experimental and control groups were 3.83 and 3.49, respectively. There was a significant difference in self-efficacy between the two groups ($F(1, 42) = 10.54, p = 0.002 < 0.05$), with a medium effect size ($\eta^2 = 0.201$). The results indicated that students using AI-CL had significantly higher self-efficacy compared to those using CSCL.

5.4 Learning interest

One-way ANCOVA was used to assess students' learning interest. The independent variable was the collaborative learning model, the dependent variable was the post-test score of learning interest, and the covariate was the pre-test score. The Levene's test was not significant ($F(1, 43) = 0.19, p > 0.05$), indicating no violation of the homogeneity of variance assumption. The results also passed the test for homogeneity of regression slopes ($F(1, 41) = 0.14, p > 0.05$). As shown in Table 5, the adjusted means for the experimental and control groups were 3.89 and 3.64, respectively. There was a significant difference in learning interest between the two groups

Table 4 The one-way ANCOVA result of self-efficacy

Group	N	Mean	SD	Adjust M	Adjust SD	F	η^2
Experimental group	24	3.89	0.52	3.83	0.77	10.54**	0.201
Control group	21	3.42	0.71	3.49	0.72		

** $p < 0.01$

Table 5 The one-way ANCOVA result of learning interest

Group	N	Mean	SD	Adjust M	Adjust SD	F	η^2
Experimental group	24	3.95	0.74	3.89	0.77	4.76*	0.102
Control group	21	3.57	0.61	3.64	0.83		

* $p < 0.05$

($F(1, 42) = 4.76$, $p = 0.035 < 0.05$), with a medium effect size ($\eta^2 = 0.102$). The results indicated that students using AI-CL had significantly higher learning interest compared to those using CSCL.

6 Discussion and conclusion

This quasi-experimental study aimed to investigate the impact of an AI-Agent-supported collaborative learning method on university students' performance in a programming course. The experimental group engaged in programming learning using the AI-CL, while the control group used the traditional CSCL. The results indicated that the AI-CL group significantly outperformed the CSCL group in terms of learning achievement, self-efficacy, and learning interest. Moreover, the AI-CL group exhibited significantly lower mental effort in cognitive load compared to the CSCL group, while no significant difference was observed between the two groups in terms of mental load.

In terms of learning achievement, learners using AI-CL performed much better than those using CSCL. This finding aligns with existing literature, which suggests that providing personalized learning experiences based on individual learner characteristics can effectively improve learning achievement (Hung and Chen, 2018). The advantages of the AI-CL environment are reflected in several aspects: firstly, the AI-Agent can provide highly personalized learning support. For slower learners, the system offers additional resources, while for faster learners, it provides more advanced guidance. This dynamic adjustment ensures that each learner can study at the most appropriate level of difficulty. Secondly, through natural dialogue interactions with the AI-Agent, the system can intelligently allocate tasks and facilitate discussions, thereby enhancing team learning efficiency. The AI-Agent's real-time feedback and adaptive guidance make the collaborative learning process smoother and more efficient (see Appendix G for details). In contrast, the CSCL environment provides a relatively fixed learning experience. Although traditional computer-supported systems can offer a certain degree of feedback, they often lack the flexibility to adjust to individual needs (Zheng et al., 2023a). For learners facing difficulties, the system may only provide pre-set prompts without offering in-depth explanations and personalized guidance as the AI-Agent does. This limitation may hinder some learners from effectively overcoming learning obstacles. The findings of this study further support the importance of integrating AI technology into educational

practice (Chiu et al., 2023). They suggest that by leveraging the advantages of AI, we can create more effective and engaging learning environments, thereby improving the quality of education and learning achievement (Jin et al., 2023).

Regarding cognitive load, students using AI-CL exhibited significantly lower cognitive load compared to those using CSCL. Further analysis revealed a significant difference between the two groups in terms of mental effort, while no significant difference was found in terms of mental load. Mental load refers to the cognitive demand placed on learners due to the requirements of the task itself (Mo et al., 2022; Skulmowski & Xu, 2022), whereas mental effort represents the actual cognitive resources and effort that learners invest to meet these task demands (Sweller et al., 2019). In other words, mental load focuses more on the cognitive demands imposed by the task, while mental effort emphasizes the learners' efforts to meet those demands. The lack of significant difference in mental load between the experimental and control groups is reasonable, as both groups faced the same task requirements and content. However, the significant difference in mental effort highlights the unique role of AI-Agent in collaborative learning. As Xiao et al. (2024) pointed out, the AI-Agent helps learners overcome their "zone of proximal development" by providing timely "learning scaffolding," thereby reducing the cognitive pressure of completing tasks. Specifically, the AI-Agent uses guiding questions and step-by-step prompts to help learners understand and solve problems more effectively, thus reducing the mental effort required to solve these problems. Overall, the significant differences in cognitive load suggest the potential advantages of the AI-CL approach in enhancing learning efficiency and reducing cognitive load.

In terms of self-efficacy, the AI-CL group demonstrated higher self-efficacy in collaborative programming compared to the CSCL group. This phenomenon can be explained from several perspectives. First, the immediacy and accuracy of AIGC tools play a key role in enhancing learners' self-efficacy. Johnson et al. (2024) noted that AI can quickly analyze code and provide solutions, and this timely assistance effectively reduces learners' frustration. Yilmaz and Yilmaz (2023) further explained that AI systems like GPT can alleviate part of the cognitive load, such as code completion and error detection, allowing learners to focus on core problems, thereby boosting self-efficacy. Second, the AI-Agent not only provides support at the code level during collaborative programming but also enhances learners' self-regulation abilities. Zimmerman's (2000) research suggests that improved self-efficacy significantly influences students' learning behavior and academic performance. Building on this, Leinonen et al. (2023) found that using LLM to translate program error messages into easily understandable natural language not only generates code explanations that are friendly to programming learners but also provides suggestions for fixing errors, thus improving self-efficacy. This AI-assisted support may encourage students to adopt more proactive learning strategies when faced with complex programming tasks, rather than avoiding them due to difficulties. This positive cycle not only boosts students' learning motivation but also enhances their confidence and resilience in the face of challenges. Cassidy (2015) pointed out that in a field like programming, which requires high levels of concentration and sustained effort, higher self-efficacy helps students better manage learning pressure and maintain consistent learning motivation and perseverance when facing challenges.

In summary, the application of AI-Agent in collaborative learning not only enhances students' immediate programming skills but also significantly improves their self-efficacy.

In terms of learning interest, students who adopt the AI-CL method show higher levels of learning interest throughout the collaborative programming process. As a key driver for continuous and deep learning, the importance of learning interest cannot be overstated. This difference is reflected in several dimensions: Firstly, AI can play the role of a virtual teacher in the collaborative learning process, continuously interacting with learners and stimulating their learning interest and motivation through positive feedback (Yu, 2023). Secondly, compared to traditional CSCL methods, the AI-CL method exhibits stronger personalized features in collaborative learning. We found that the AI-CL method can dynamically adjust learning content and methods based on the individual needs and ability levels of learners, improving the alignment of the learning experience. This finding is consistent with the research of Fuchs (2023), which shows that personalized learning significantly enhances learner engagement and learning interest. Furthermore, in actual programming practice, the AI agent can immediately detect errors and provide targeted feedback, helping students quickly locate and correct problems. This real-time feedback not only accelerates students' mastery of programming skills but also boosts their learning confidence through accumulated successful experiences. The study further found that the real-time feedback and feedforward mechanisms of the AI agent played a significant role in enhancing collaborative knowledge construction, co-regulation behaviors, and team outcomes (Zheng et al., 2023b). In summary, the AI agent, through intelligent role-playing, adaptive learning guidance, real-time interactive feedback, and collaborative behavior guidance, effectively improves students' learning interest and engagement in programming courses. This positive impact not only helps students achieve better results in the current course but may also have a profound influence on their future learning paths and career development.

Although this study supports the effectiveness of the AI-CL method, there are some limitations to consider. First, this study focused on the impact of the equipment environment and course settings, but the sample size was not large enough. Therefore, further discussion is needed on whether the results can be generalized to other courses. Second, although researchers set certain restrictions on the AI-Agent to avoid directly providing answers to learners, in practice, learners might still receive more detailed responses from the AI-Agent than necessary. This could result in the AI-Agent providing overly comprehensive information, which may hinder learners' active absorption and processing of knowledge. Moreover, the current AI-Agent is limited by the capabilities of large language models. While these models can already facilitate a considerable degree of intelligent interaction, they still have limitations such as a constrained context window and issues with long-term memory retention (Wang et al., 2024c). In the future, we plan to integrate more advanced AI technologies into the AI-CL method, such as multimodal learning, knowledge graphs, and causal reasoning, to provide more personalized and adaptive learning support. The AI-CL method used in this study also warrants further refinement and enhancement. Additionally, we aim to conduct longer-term experiments to further investigate the effectiveness of the proposed method across various dimensions.

In conclusion, based on the experimental results, the AI-CL method developed in this study helped learners achieve better learning achievement, higher self-efficacy, and increased learning interest in the programming course, while also reducing mental effort in cognitive load. The AI-Agent had a positive impact on university students' programming learning in collaborative settings. Although there was no significant difference in mental load between the two groups, the AI-CL method demonstrated clear advantages in other areas. It is hoped that the findings of this study will provide valuable insights for future AI-based educational technology development and the design of collaborative learning models.

7 Implication

7.1 Theoretical implications

This study provides new insights into the application of cognitive load theory in AI-supported learning environments, particularly with the significant differences observed in the dimension of mental effort. This finding not only expands the scope of application for cognitive load theory but also lays a solid theoretical foundation for further optimizing cognitive processing in AI-supported learning environments. Additionally, the results indicate that AI-Agent can significantly enhance learners' self-efficacy in collaborative learning, enriching Bandura's self-efficacy theory, especially in terms of the mechanisms of self-efficacy formation in technology-enhanced learning environments. The instant feedback and personalized support provided by AI-Agent may become a new way to influence self-efficacy. Through this, individuals can receive timely encouragement and positive feedback, thereby enhancing their confidence in their abilities. This opens up new research directions for the application of self-efficacy theory in digital learning environments.

7.2 Practical implications

On a practical level, the successful application of the AI-CL method provides higher education institutions with an innovative teaching model. Universities may consider introducing AI-Agent-supported collaborative learning method in technology-intensive courses like programming to effectively integrate AI technology with traditional teaching methods, thus creating a more dynamic and personalized learning environment. The findings emphasize the crucial role of AI-Agent in providing personalized learning support, pointing the way for educational technology developers. They should focus on the adaptive functions of AI-Agent to maximize learning achievement while maintaining an appropriate level of cognitive challenge to promote deep learning. Although AI-Agent has demonstrated excellence in enhancing learning achievement, educators must remain cautious about the potential risks of over-reliance on AI. In practical teaching, a balance should be struck between AI assistance and human guidance to ensure that students not only effectively utilize AI tools but also continue to develop critical thinking and independent problem-solving skills, laying the foundation for lifelong learning in the future.

Appendix A. Basic ability test questionnaire

1. In data structures, the characteristic of a stack is:
 - A. First In First Out (FIFO)
 - B. First In Last Out (LIFO)
 - C. Random Access
 - D. Last In First Out (FILO)
2. Which of the following data structures is linear?
 - A. Tree
 - B. Graph
 - C. Stack
 - D. Heap
3. Which of the following is a valid sequence of operations for a stack?
 - A. Push 1—> Push 2—> Pop 2—> Pop 1
 - B. Push 1—> Push 2—> Pop 1—> Pop 2
 - C. Push 2—> Push 1—> Pop 1—> Pop 2
 - D. Push 1—> Pop 2—> Push 2—> Pop 1
4. In which of the following scenarios is a stack most suitable?
 - A. Finding the shortest path
 - B. Implementing recursion
 - C. Finding cycles in a graph
 - D. Sorting
5. Recursion typically utilizes which auxiliary data structure?
 - A. Queue
 - B. Stack
 - C. Array
 - D. Linked List
6. Which statement about queues is correct?
 - A. A queue is Last In First Out (LIFO)
 - B. A queue is First In First Out (FIFO)
 - C. A queue allows random access
 - D. A queue can only store integers

7. Which of the following is not a programming language?
 - A. Python
 - B. Java
 - C. C++
 - D. HTML
8. Which of the following is not a common data structure?
 - A. Tree
 - B. Heap
 - C. Algorithm
 - D. Queue
9. In C++, which keyword is used to define a constant?
 - A. int
 - B. const
 - C. define
 - D. static
10. Which of the following stack operations could cause a stack overflow?
 - A. Pushing beyond the stack's maximum capacity
 - B. Popping until the stack is empty
 - C. Pushing when there are no elements
 - D. Popping until the stack is full
11. Which data structure is best suited for implementing an 'undo' feature?
 - A. Stack
 - B. Queue
 - C. Array
 - D. Hash Table
12. The time complexity to find an element in an array is:
 - A. $O(1)$
 - B. $O(\log n)$
 - C. $O(n)$
 - D. $O(n^2)$
13. Which operator is used to take the address in C++?
 - A. &

- B. *
 - C. &&
 - D. ||
14. Which of the following sorting algorithms has an average time complexity of $O(n^2)$?
- A. Quick Sort
 - B. Bubble Sort
 - C. Heap Sort
 - D. Merge Sort
15. Which of the following is not a basic characteristic of algorithms?
- A. Determinism
 - B. Finiteness
 - C. Randomness
 - D. Input and Output
16. Which programming paradigm emphasizes the separation of data and operations?
- A. Object-Oriented Programming
 - B. Procedural Programming
 - C. Functional Programming
 - D. Service-Oriented Programming
17. In Python, which of the following data structures is immutable?
- A. List
 - B. Set
 - C. Dictionary
 - D. Tuple
18. In computing, the binary number 1101 equals what decimal number?
- A. 11
 - B. 12
 - C. 13
 - D. 14
19. In C++, which data type is used to store a single character?
- A. int
 - B. char

- C. float
- D. string

20. In Python, which symbol is used to indicate a comment?

- A. //
- B. #
- C. *
- D. #

Appendix B. Cognitive load questionnaire

Mental load

1. The content of this programming task is very difficult for me.
2. I have to put a lot of effort into solving this programming task.
3. Solving the problem in this programming task is very troublesome for me.
4. I feel very frustrated when solving the problem in this programming task.
5. I don't have enough time to complete all the tasks in this programming task.

Mental effort

1. During programming tasks, the presentation of task requirements or programming content requires me to put in a lot of cognitive effort.
2. I need to put in a lot of effort to complete all the requirements or achieve the expected goals in this programming task.
3. It is difficult to keep up with and understand the requirements and guidance in this programming task.

Appendix C. Self-efficacy questionnaire

1. I believe that I can successfully complete programming tasks.
2. Even when faced with difficulties in programming tasks, I am able to find a solution to the problem.
3. When facing setbacks in collaborative programming, I am able to quickly adjust and continue moving forward.
4. I believe that I can effectively complete team collaboration tasks in collaborative programming.

Appendix D. learning interest questionnaire

1. Is the theme covered by this programming task exciting to me?
2. My interest in learning programming has increased.
3. The task of collaborative programming is very interesting and beneficial for my learning.
4. I am very interested in the learning content
5. The learning activity experience feels very positive.

Appendix E. Task sheet example

Task Sheet

[Task Scenario]:

You need to read and solve the following issues, and finally submit a copy of the code to the platform.

[Description]:

Imagine you are managing a to-do list application where tasks are organized in a stack-like manner. You want to implement a stack that supports the following operations:

push (x): Add a task x to the to-do list. For example, push("Buy groceries") adds "Buy groceries" to the list.

Pop(): Complete the most recent task. If there are no tasks left to complete, output 'Empty' to indicate that the to-do list is empty.

query(): Check the most recent task that needs to be completed. If the to-do list is empty, output 'Anguei!' to signal that there are no pending tasks.

size(): Output the number of tasks currently in the to-do list. This helps in tracking how many tasks are yet to be completed.

For example, after pushing tasks "Write report," "Call client," and "Buy groceries," the stack operations will ensure you address "Buy groceries" first when popping tasks, and querying will keep you informed of your next immediate task.

[Input Format]:

There are multiple sets of data within the testing point of this questionnaire.

The first line of input is an integer T, representing the number of data groups. For each set of data, the format is as follows:

The first row of each set of data is an integer representing the number of operations n.

Next, in the n line, each line is first composed of a string, which is one of 'push', 'pop', 'query', and 'size'. If it is 'push', then there is an integer x after it, representing the number to be added, separated by a space between x and the string; If it is not a 'push', then there is no other content in this line.

[Output Format]:

For each group of data, output them in order according to the requirements in the "Description". Each output occupies one line.

Appendix F. code

Code1. Control group code example

Import sys

```
# input = sys.stdin.readline().strip()
t = int(input().strip())
for _ in range(t):
    st = []
    n = int(input().strip())
    for i in range(n):
        ope = input().strip()
        if ope[:2] == "pu":
            o, num = ope.split()
            st.append(num)
        elif ope[:2] == "po":
            if not len(st):
                print("Empty")
            else:
                temp = st.pop()
                # print(temp)
        elif ope[:2] == "qu":
            if not len(st):
                print("Anguei!")
            else:
                temp = st.pop()
                print(temp)
                st.append(temp)
        elif ope[:2] == "si":
            print(len(st))
```

Code2. Experimental group code example

Class Stack:

```
def __init__(self):
    self.items = []
def push(self, item):
    self.items.append(item)
def pop(self):
    if not self.is_empty():
        self.items.pop()
        return "None"
    else:
        return "Empty"
def query(self):
    if not self.is_empty():
```

```

return self.items[-1]
else:
return "Anguei!"
def is_empty(self):
return len(self.items) == 0
def size(self):
return len(self.items)
def __repr__(self):
return f"Stack({self.items})"
def process_operations(operations):
stack = Stack()
results = []
For operation in operations:
if operation.startswith("push"):
_, x = operation.split()
stack.push(int(x))
elif operation == "pop":
# results.append(stack.pop())
if stack.pop() == "Empty":
results.append("Empty")
elif operation == "query":
results.append(stack.query())
elif operation == "size":
results.append(stack.size())
return results
def main():
import sys
input = sys.stdin.read
data = input().splitlines()
T = int(data[0])
index = 1
for _ in range(T):
n = int(data[index])
operations = data[index + 1:index + 1 + n]
results = process_operations(operations)
for result in results:
print(result)
index += 1 + n # Update the index to the position of the next set of data
# Call the main function
if __name__ == "__main__":
main()

```

Appendix G. The interaction process between students and AI-agent(Part)

[User]:

You are a senior programmer skilled in solving common algorithm problems, and now you will be using Python language to solve a programming problem. Please note that when solving the problem, you need to follow the prompts I provided step by step.

[AI-Agent]:

Okay, let's solve a programming problem. Please tell me the specific problem and corresponding tips, and I will follow the steps to help you solve it.

[User]:

This problem requires you to implement a stack.

[AI-Agent]:

Okay, let's implement a simple stack data structure. Stack is a linear data structure that follows the Last In First Out (LIFO) principle and mainly supports the following operations:

Push (x): Push element x onto the stack.

Pop(): Remove and return the element at the top of the stack.

Top(): Returns the element at the top of the stack.

Is_empty(): Returns whether the stack is empty.

Next, we will implement this stack step by step according to the prompts.

Step 1: Define stack class

Firstly, we need to define a stack class and initialize its data structure. We can use lists to implement stack storage.

```
class Stack:
    def __init__(self):
        self.items = []
    def __repr__(self):
        return f"Stack({self.items})"
```

Step 2: Implement the push method

Next, we implement the push method to add elements to the top of the stack.

```
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def __repr__(self):
        return f"Stack({self.items})"
```

Step 3: Implement the pop method

Then, we implement the pop method to remove and return elements from the top of the stack.

```
class Stack:
    def __init__(self):
        self.items = []
    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("pop from empty stack")
    def __repr__(self):
        return f"Stack({self.items})"
```

Step 4: Implement the top method

Next, we implement the top method, which returns the element at the top of the stack but does not remove it.

```
class Stack:
    def __init__(self):
        self.items = []
    def top(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("top from empty stack")
    def __repr__(self):
        return f"Stack({self.items})"
```

Step 5: Implement the is_empty method

Finally, we implement the is_empty method to check if the stack is empty.

```
class Stack:
    def __init__(self):
        self.items = []
    def is_empty(self):
        return len(self.items) == 0
    def __repr__(self):
        return f"Stack({self.items})"
```

Test our stack: Finally, let's test the implementation of this stack.

```
if __name__ == "__main__":
    stack = Stack()
    print(stack.is_empty()) # True.
    stack.push(1).
    stack.push(2).
    stack.push(3).
    print(stack) # Stack([1, 2, 3]).
```

```

print(stack.top()) # 3.
print(stack.pop()) # 3.
print(stack) # Stack([1, 2]).
print(stack.is_empty()) # False.
stack.pop().
stack.pop().
print(stack.is_empty()) # True.
try:
stack.pop().
except IndexError as e:
print(e) # pop from empty stack.

```

This code defines a stack class and tests the functionality of all methods. You can run this code to verify if our stack implementation is correct.

Data availability The datasets generated during and/or analysed during the current study are available from the corresponding author on reasonable request.

Declarations

Conflict of interest During the research, the authors indicate that no commercial or financial ties that may be regarded a possible conflict of interest existed.

References

- Ackerman, R. (2019). Heuristic cues for meta-reasoning judgments: Review and methodology. *Psihologijske Teme*, 28(1), 1–20. <https://doi.org/10.31820/pt.28.1.1>
- Allcoat, D., & von Mühlenen, A. (2018). Learning in virtual reality: Effects on performance, emotion and engagement. *Research in Learning Technology*, 26. <https://doi.org/10.25304/rlt.v26.2140>
- Armoni, M., Meerbaum-Salant, O., & Ben-Ari, M. (2015). From scratch to “real” programming. *ACM Transactions on Computing Education (TOCE)*, 14(4), 1–15. <https://doi.org/10.1145/2677087>
- Bandura, A., & Wessels, S. (1997). *Self-efficacy* (pp. 4–6). Cambridge University Press.
- Bansal, G., Chamola, V., Hussain, A., Guizani, M., & Niyato, D. (2024). Transforming conversations with AI—a comprehensive study of ChatGPT. *Cognitive Computation*, 1–24. <https://doi.org/10.1007/s12559-023-10236-2>
- Baucal, A., Jošić, S., Ilić, I. S., Videnović, M., Ivanović, J., & Krstić, K. (2023). What makes peer collaborative problem solving productive or unproductive: A qualitative systematic review. *Educational Research Review*, 100567. <https://doi.org/10.1016/j.edurev.2023.100567>
- Belland, B. R., Walker, A. E., Kim, N. J., & Lefler, M. (2017). Synthesizing results from empirical research on computer-based scaffolding in STEM education: A meta-analysis. *Review of Educational Research*, 87(2), 309–344. <https://doi.org/10.3102/0034654316670999>
- Benítez, T. M., Xu, Y., Boudreau, J. D., Kow, A. W. C., Bello, F., Van Phuoc, L., ... & Chung, K. C. (2024). Harnessing the potential of large language models in medical education: promise and pitfalls. *Journal of the American Medical Informatics Association*, 31(3), 776–783. <https://doi.org/10.1093/jamia/ocad252>
- Buchner, J., Buntins, K., & Kerres, M. (2022). The impact of augmented reality on cognitive load and performance: A systematic review. *Journal of Computer Assisted Learning*, 38(1), 285–303. <https://doi.org/10.1111/jcal.12617>
- Capdeferro, N., & Romero, M. (2012). Are online learners frustrated with collaborative learning experiences? *International Review of Research in Open and Distributed Learning*, 13(2), 26–44. <https://doi.org/10.19173/irrodl.v13i2.1127>

- Cassidy, S. (2015). Resilience building in students: The role of academic self-efficacy. *Frontiers in Psychology*, 6, 1781. <https://doi.org/10.3389/fpsyg.2015.01781>
- Chen, L., Chen, P., & Lin, Z. (2020). Artificial intelligence in education: A review. *IEEE Access*, 8, 75264–75278. <https://doi.org/10.1109/ACCESS.2020.2988510>
- Chen, X. J., Hu, Z. B., & Wang, C. L. (2024). Empowering education development through AIGC: A systematic literature review. *Education and Information Technologies*, 29, 17485–17537. <https://doi.org/10.1007/s10639-024-12549-7>
- Chen, X. J., Yu, T., Jian, D., Jing, Y. H. & Wang, C. L. (2025). Unveiling Learners' Intentions toward Influencer-led Education: An Integration of Qualitative and Quantitative Analysis. *Interactive Learning Environments*, 1–19. <https://doi.org/10.1080/10494820.2024.2444533>
- Chi, M. T., & Wylie, R. (2014). The ICAP framework: Linking cognitive engagement to active learning outcomes. *Educational Psychologist*, 49(4), 219–243. <https://doi.org/10.1080/00461520.2014.965823>
- Chiu, T. K., Moorhouse, B. L., Chai, C. S., & Ismailov, M. (2023). Teacher support and student motivation to learn with Artificial Intelligence (AI) based chatbot. *Interactive Learning Environments*, 1–17. <https://doi.org/10.1080/10494820.2023.2172044>
- Cress, U., & Kimmerle, J. (2023). Co-constructing knowledge with generative AI tools: Reflections from a CSCL perspective. *International Journal of Computer-Supported Collaborative Learning*, 18(4), 607–614. <https://doi.org/10.1007/s11412-023-09409-w>
- Dai, J., Zhang, X., & Wang, C. L. (2024). A meta-analysis of learners' continuance intention toward online education platforms. *Education and Information Technologies*, 29, 21833–21868. <https://doi.org/10.1007/s10639-024-12654-7>
- Estácio, B., Oliveira, R., Marczak, S., Kalinowski, M., Garcia, A., Prikladnicki, R., & Lucena, C. (2015, September). Evaluating collaborative practices in acquiring programming skills: Findings of a controlled experiment. In *2015 29th Brazilian Symposium on Software Engineering* (pp. 150–159). IEEE. <https://doi.org/10.1109/SBES.2015.24>
- Fang, J. W., Shao, D., Hwang, G. J., & Chang, S. C. (2022). From critique to computational thinking: A peer-assessment-supported problem identification, flow definition, coding, and testing approach for computer programming instruction. *Journal of Educational Computing Research*, 60(5), 1301–1324. <https://doi.org/10.1177/07356331211060470>
- Fogli, D., Gargioni, L., Guida, G., & Tampalini, F. (2022). A hybrid approach to user-oriented programming of collaborative robots. *Robotics and Computer-Integrated Manufacturing*, 73, 102234. <https://doi.org/10.1016/j.rcim.2021.102234>
- Fuchs, K. (2023). Exploring the opportunities and challenges of NLP models in higher education: Is Chat GPT a blessing or a curse? In *Frontiers in Education*, 8, 1166682. <https://doi.org/10.3389/feduc.2023.1166682>. Frontiers.
- Ghanizadeh, A. (2017). The interplay between reflective thinking, critical thinking, self-monitoring, and academic achievement in higher education. *Higher Education*, 74, 101–114. <https://doi.org/10.1007/s10734-016-0031-y>
- Graesser, A. C., Fiore, S. M., Greiff, S., Andrews-Todd, J., Foltz, P. W., & Hesse, F. W. (2018). Advancing the science of collaborative problem solving. *Psychological Science in the Public Interest*, 19(2), 59–92. <https://doi.org/10.1177/152910061880824>
- Hernández-Sellés, N., Muñoz-Carril, P. C., & González-Sanmamed, M. (2019). Computer-supported collaborative learning: An analysis of the relationship between interaction, emotional support and online collaborative tools. *Computers & Education*, 138, 1–12. <https://doi.org/10.1016/j.compedu.2019.04.012>
- Higashi, R. M., Schunn, C. D., & Flot, J. B. (2017). Different underlying motivations and abilities predict student versus teacher persistence in an online course. *Educational Technology Research and Development*, 65, 1471–1493. <https://doi.org/10.1007/s11423-017-9528-z>
- Hoch, E., Sidi, Y., Ackerman, R., Hoogerheide, V., & Scheiter, K. (2023). Comparing mental effort, difficulty, and confidence appraisals in problem-solving: A metacognitive perspective. *Educational Psychology Review*, 35(2), 61. <https://doi.org/10.1007/s10648-023-09779-5>
- Hudon, A., Demazure, T., Karran, A., Léger, P. M., & Sénécal, S. (2021). Explainable artificial intelligence (XAI): how the visualization of AI predictions affects user cognitive load and confidence. In *Information Systems and Neuroscience: NeuroIS Retreat 2021* (pp. 237–246). Springer International Publishing. https://doi.org/10.1007/978-3-030-88900-5_27
- Hung, I. C., & Chen, N. S. (2018). Embodied interactive video lectures for improving learning comprehension and retention. *Computers & Education*, 117, 116–131. <https://doi.org/10.1016/j.compedu.2017.10.005>

- Hwang, G. J., Yang, L. H., & Wang, S. Y. (2013). A concept map-embedded educational computer game for improving students' learning performance in natural science courses. *Computers & Education*, 69, 121–130. <https://doi.org/10.1016/j.compedu.2013.07.008>
- Hwang, G. J., Sung, H. Y., Chang, S. C., & Huang, X. C. (2020). A fuzzy expert system-based adaptive learning approach to improving students' learning performances by considering affective and cognitive factors. *Computers and Education: Artificial Intelligence*, 1, 100003. <https://doi.org/10.1016/j.caeai.2020.100003>
- Iku-Silan, A., Hwang, G. J., & Chen, C. H. (2023). Decision-guided chatbots and cognitive styles in interdisciplinary learning. *Computers & Education*, 201, 104812. <https://doi.org/10.1016/j.compedu.2023.104812>
- Janssen, J., & Kirschner, P. A. (2020). Applying collaborative cognitive load theory to computer-supported collaborative learning: Towards a research agenda. *Educational Technology Research and Development*, 68(2), 783–805. <https://doi.org/10.1007/s11423-019-09729-5>
- Järvelä, S., & Hadwin, A. F. (2013). New frontiers: Regulating learning in CSCL. *Educational Psychologist*, 48(1), 25–39. <https://doi.org/10.1080/00461520.2012.748006>
- Jeong, H., Hmelo-Silver, C. E., & Jo, K. (2019). Ten years of computer-supported collaborative learning: A meta-analysis of CSCL in STEM education during 2005–2014. *Educational Research Review*, 28, 100284. <https://doi.org/10.1016/j.edurev.2019.100284>
- Jin, S. H., Im, K., Yoo, M., Roll, I., & Seo, K. (2023). Supporting students' self-regulated learning in online learning using artificial intelligence applications. *International Journal of Educational Technology in Higher Education*, 20(1), 37. <https://doi.org/10.1186/s41239-023-00406-5>
- Jin, F., Peng, X., Sun, L., Song, Z., Zhou, K., & Lin, C.-H. (2025). Knowledge (co-)construction among artificial intelligence, novice teachers, and experienced teachers in an online professional learning community. *Journal of Computer Assisted Learning*, 41(2). <https://doi.org/10.1111/jcal.70004>
- Jin, F., Sun, L., Pan, Y., & Lin, C. H. (2025a). High Heels, Compass, Spider-Man, or Drug? Metaphor Analysis of Generative Artificial Intelligence in Academic Writing. *Computers & Education*, 105248. <https://doi.org/10.1016/j.compedu.2025.105248>
- Jing, Y. H., Wang, C. L., Chen, Z. Y., Shen, S. S., & Shadiev, R. (2024a). A Bibliometric analysis of studies on technology-supported learning environments: Hot topics and frontier evolution. *Journal of Computer Assisted Learning*, 40(3), 1185–1200. <https://doi.org/10.1111/jcal.12934>
- Jing, Y. H., Wang, H. M., Chen, X. J., & Wang, C. L. (2024b). What factors will affect the effectiveness of using ChatGPT to solve programming problems? A quasi-experimental study. *Humanities & Social Sciences Communications*, 11, 319. <https://doi.org/10.1057/s41599-024-02751-w>
- Johnson, D. M., Doss, W., & Estepp, C. M. (2024). Using ChatGPT with novice arduino programmers: Effects on performance, interest, self-efficacy, and programming ability. *Journal of Research in Technical Careers*, 8(1), 1. <https://doi.org/10.9741/2578-2118.1152>
- Kalelioglu, F. (2015). A new way of teaching programming skills to K-12 students: Code. org. *Computers in Human Behavior*, 52, 200–210. <https://doi.org/10.1016/j.chb.2015.05.047>
- Kimmel, B., Geisert, A. L., Yaro, L., Gipson, B., Hotchkiss, R. T., Osaé-Asante, S. K., ... & Yamaguchi, C. (2024, May). Enhancing Programming Error Messages in Real Time with Generative AI. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (pp. 1–7). <https://doi.org/10.1145/3613905.3647967>
- Kirschner, P. A., Sweller, J., Kirschner, F., & Zambrano, R. J. (2018). From cognitive load theory to collaborative cognitive load theory. *International Journal of Computer-Supported Collaborative Learning*, 13, 213–233. <https://doi.org/10.1007/s11412-018-9277-y>
- Laak, K. J., & Aru, J. (2024). AI and personalized learning: bridging the gap with modern educational goals. arXiv preprint arXiv:2404.02798. <https://doi.org/10.48550/arXiv.2404.02798>
- Lee, J. S. (2014). The relationship between student engagement and academic performance: Is it a myth or reality? *The Journal of Educational Research*, 107(3), 177–185. <https://doi.org/10.1080/00220671.2013.807491>
- Leinonen, J., Hellas, A., Sarsa, S., Reeves, B., Denny, P., Prather, J., & Becker, B. A. (2023). March). Using large language models to enhance programming error messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education*, 1, 563–569. <https://doi.org/10.1145/3545945.3569770>
- Lin, Y. P., & Yu, Z. G. (2024). A bibliometric analysis of artificial intelligence chatbots in educational contexts. *Interactive Technology and Smart Education*, 21(2), 189–213. <https://doi.org/10.1108/ITSE-12-2022-0165>

- Lin, G. Y., Liao, Y. W., Su, Z. Y., Wang, Y. M., & Wang, Y. S. (2023a). What drives undergraduates' effort and persistence in learning programming. *Education and Information Technologies*, 28(10), 12383–12406. <https://doi.org/10.1007/s10639-023-11670-3>
- Lin, Y. P., Niu, R. H., & Yu, Z. G. (2023). Roles of ambiguity tolerance and learning effectiveness: Structural equation modeling evidence from EFL students' perceptions of factors influencing peer collaboration. *Language Teaching Research*, 13621688231216201. <https://doi.org/10.1177/13621688231216201>
- Lindberg, R. S., Laine, T. H., & Haaranen, L. (2019). Gamifying programming education in K-12: A review of programming curricula in seven countries and programming games. *British Journal of Educational Technology*, 50(4), 1979–1995. <https://doi.org/10.1111/bjet.12685>
- Liu, H., Wu, Z., Lu, Y., & Zhu, L. (2022). Exploring the balance between computational thinking and learning motivation in elementary programming education: An empirical study with game-based learning. *IEEE Transactions on Games*, 15(1), 95–107. <https://doi.org/10.1109/TG.2022.3143701>
- Makransky, G., & Mayer, R. E. (2022). Benefits of taking a virtual field trip in immersive virtual reality: Evidence for the immersion principle in multimedia learning. *Educational Psychology Review*, 34(3), 1771–1798. <https://doi.org/10.1007/s10648-022-09675-4>
- Maples, B., Pea, R. D., & Markowitz, D. (2023). Learning from intelligent social agents as social and intellectual mirrors. *Learning: Designing the Future*, 73. https://doi.org/10.1007/978-3-031-09687-7_5
- Mazer, J. P. (2013). Validity of the student interest and engagement scales: Associations with student learning outcomes. *Communication Studies*, 64(2), 125–140. <https://doi.org/10.1080/10510974.2012.727943>
- Mo, C. Y., Wang, C. L., Dai, J., & Jin, P. Q. (2022). Video playback speed influence on learning effect from the perspective of personalized adaptive learning: A study based on cognitive load theory. *Frontiers in Psychology*, 13, 839982. <https://doi.org/10.3389/fpsyg.2022.839982>
- Nawahdah, M., & Taji, D. (2016). Investigating students' behavior and code quality when applying pair-programming as a teaching technique in a Middle Eastern society. In *2016 IEEE Global Engineering Education Conference (EDUCON)* (pp. 32–39). IEEE. <https://doi.org/10.1109/EDUCON.2016.7474527>
- Ouyang, F., Xu, W., & Cukurova, M. (2023). An artificial intelligence-driven learning analytics method to examine the collaborative problem-solving process from the complex adaptive systems perspective. *International Journal of Computer-Supported Collaborative Learning*, 18(1), 39–66. <https://doi.org/10.1007/s11412-023-09387-z>
- Pirzado, F. A., Ahmed, A., Mendoza-Urdiales, R. A., & Terashima-Marin, H. (2024). Navigating the pitfalls: Analyzing the behavior of LLMs as a coding assistant for computer science students—a systematic review of the literature. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2024.3443621>
- Renninger, K. A., & Hidi, S. (2011). Revisiting the conceptualization, measurement, and generation of interest. *Educational Psychologist*, 46(3), 168–184. <https://doi.org/10.1080/00461520.2011.587723>
- Rienties, B., K hler Simonsen, H., & Herodotou, C. (2020). Defining the boundaries between artificial intelligence in education, computer-supported collaborative learning, educational data mining, and learning analytics: A need for coherence. In *Frontiers in Education* 5 128. Frontiers Media SA. <https://doi.org/10.3389/feduc.2020.00128>
- Rowbotham, M., & Schmitz, G. S. (2013). Development and validation of a student self-efficacy scale. *Journal of Nursing & Care*, 2(1), 1–6. <https://doi.org/10.4172/2167-1168.1000126>
- Salleh, N., Mendes, E., & Grundy, J. (2010). Empirical studies of pair programming for CS/SE teaching in higher education: A systematic literature review. *IEEE Transactions on Software Engineering*, 37(4), 509–525. <https://doi.org/10.1109/TSE.2010.59>
- Skulmowski, A., & Xu, K. M. (2022). Understanding cognitive load in digital and online learning: A new perspective on extraneous cognitive load. *Educational Psychology Review*, 34(1), 171–196. <https://doi.org/10.1007/s10648-021-09624-7>
- Smiderle, R., Rigo, S. J., Marques, L. B., de Miranda, P., Coelho, J. A., & Jaques, P. A. (2020). The impact of gamification on students' learning, engagement and behavior based on their personality traits. *Smart Learning Environments*, 7(1), 3. <https://doi.org/10.1186/s40561-019-0098-x>
- Su, N., Al Mamun, A., Reza, M. N. H., Yang, Q., & Masud, M. M. (2024). Unveiling the nexus between quality and student engagement in web-based collaborative learning systems. *Education and Information Technologies*, 1–36. <https://doi.org/10.1007/s10639-024-12794-w>
- Sun, J. C. Y., & Hsu, K. Y. C. (2019). A smart eye-tracking feedback scaffolding approach to improving students' learning self-efficacy and performance in a C programming course. *Computers in Human Behavior*, 95, 66–72. <https://doi.org/10.1016/j.chb.2019.01.036>

- Sweller, J. (1994). Cognitive load theory, learning difficulty, and instructional design. *Learning and Instruction*, 4(4), 295–312. [https://doi.org/10.1016/0959-4752\(94\)90003-5](https://doi.org/10.1016/0959-4752(94)90003-5)
- Sweller, J., van Merriënboer, J. J., & Paas, F. (2019). Cognitive architecture and instructional design: 20 years later. *Educational Psychology Review*, 31, 261–292. <https://doi.org/10.1007/s10648-019-09465-5>
- Tsai, C. Y. (2019). Improving students' understanding of basic programming concepts through visual programming language: The role of self-efficacy. *Computers in Human Behavior*, 95, 224–232. <https://doi.org/10.1016/j.chb.2018.11.038>
- Tsai, C. Y., Chen, Y. A., Hsieh, F. P., Chuang, M. H., & Lin, C. L. (2024). Effects of a programming course using the GAME model on undergraduates' self-efficacy and basic programming concepts. *Journal of Educational Computing Research*, 62(3), 702–724. <https://doi.org/10.1177/07356331231206071>
- Van Leeuwen, A., Rummel, N., & Van Gog, T. (2019). What information should CSCL teacher dashboards provide to help teachers interpret CSCL situations? *International Journal of Computer-Supported Collaborative Learning*, 14, 261–289. <https://doi.org/10.1007/s11412-019-09299-x>
- Wang, X. M., & Hwang, G. J. (2017). A problem posing-based practicing strategy for facilitating students' computer programming skills in the team-based learning mode. *Educational Technology Research and Development*, 65, 1655–1671. <https://doi.org/10.1007/s11423-017-9551-0>
- Wang, C. L., Dai, J., Zhu, K. K., Yu, T., & Gu, X. Q. (2023). Understanding the Continuance Intention of College Students Toward New E-learning Spaces Based on an Integrated Model of the TAM and TTF. *International Journal of Human-Computer Interaction*, 40(24), 8419–8432. <https://doi.org/10.1080/10447318.2023.2291609>
- Wang, C. L., Chen, X. J., Yu, T., Liu, Y. D., & Jing, Y. H. (2024). Education reform and change driven by digital technology: A bibliometric study from a global perspective. *Humanities & Social Sciences Communications*, 11, 256. <https://doi.org/10.1057/s41599-024-02717-y>
- Wang, X., Zhong, Y., Huang, C., & Huang, X. (2024). ChatPRCS: A personalized support system for english reading comprehension based on ChatGPT. *IEEE Transactions on Learning Technologies*. <https://doi.org/10.1109/TLT.2024.3405747>
- Wang, C. L., Chen, X. J., Hu, Z. B., Jin, S., & Gu, X. Q. (2025). Deconstructing university learners' adoption intention towards AIGC technology: A mixed-methods study using ChatGPT as an example. *Journal of Computer Assisted Learning*, 41, e13117. <https://doi.org/10.1111/jcal.13117>
- Wang, C. L., Wang, H. M., Li Y. Y., Dai, J., Gu, X. Q. & Yu, T. (2024). Factors Influencing University Students' Behavioral Intention to Use Generative Artificial Intelligence: Integrating the Theory of Planned Behavior and AI Literacy. *International Journal of Human-Computer Interaction*. 1–23. <https://doi.org/10.1080/10447318.2024.2383033>
- Wang, C. L., Chen, X. J., Li Y. F., Wang, P. J., Wang, H. M., & Li, Y. Y. (2025). MetaClassroom: A new paradigm and experience for programming education. *Journal of Educational Computing Research*, 1–38. <https://doi.org/10.1177/07356331251322470>
- Wei, X., Lin, L., Meng, N., Tan, W., & Kong, S. C. (2021). The effectiveness of partial pair programming on elementary school students' computational thinking skills and self-efficacy. *Computers & Education*, 160, 104023. <https://doi.org/10.1016/j.compedu.2020.104023>
- Xiao, R., Hou, X., & Stamper, J. (2024). Exploring How Multiple Levels of GPT-Generated Programming Hints Support or Disappoint Novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (pp. 1–10). <https://doi.org/10.1145/3613905.3650937>
- Yildiz Durak, H. (2018). Digital story design activities used for teaching programming effect on learning of programming concepts, programming self-efficacy, and participation and analysis of student experiences. *Journal of Computer Assisted Learning*, 34(6), 740–752. <https://doi.org/10.1111/jcal.12281>
- Yilmaz, R., & Yilmaz, F. G. K. (2023). The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation. *Computers and Education: Artificial Intelligence*, 4, 100147. <https://doi.org/10.1016/j.caeai.2023.100147>
- Yu, H. (2023). Reflection on whether Chat GPT should be banned by academia from the perspective of education and teaching. *Frontiers in Psychology*, 14, 1181712. <https://doi.org/10.3389/fpsyg.2023.1181712>
- Zawacki-Richter, O., Marín, V. I., Bond, M., & Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education—where are the educators? *International Journal of Educational Technology in Higher Education*, 16(1), 1–27. <https://doi.org/10.1186/s41239-019-0171-0>

- Zhang, S., Zhao, X., Zhou, T., & Kim, J. H. (2024). Do you have AI dependency? The roles of academic self-efficacy, academic stress, and performance expectations on problematic AI usage behavior. *International Journal of Educational Technology in Higher Education*, 21(1), 34. <https://doi.org/10.1186/s41239-024-00467-0>
- Zhang, I. (Yunyi), Jia, Y. (Ruby), Cheng, X. (Alicia), Son, J. Y., & Stigler, J. W. (2025). Empowering Novice Programmers: Redesigning Documentation for Effective Informal Learning. *Journal of Educational Computing Research*. <https://doi.org/10.1177/07356331241311505>
- Zheng, L., Long, M., Niu, J., & Zhong, L. (2023). An automated group learning engagement analysis and feedback approach to promoting collaborative knowledge building, group performance, and socially shared regulation in CSCL. *International Journal of Computer-Supported Collaborative Learning*, 18(1), 101–133. <https://doi.org/10.1007/s11412-023-09386-0>
- Zheng, L., Fan, Y., Chen, B., Huang, Z., LeiGao, & Long, M. (2023). An AI-enabled feedback-feedforward approach to promoting online collaborative learning. *Education and Information Technologies*, 1–22. <https://doi.org/10.1007/s10639-023-12292-5>
- Zimmerman, B. J. (2000). Self-efficacy: An essential motive to learn. *Contemporary Educational Psychology*, 25(1), 82–91. <https://doi.org/10.1006/ceps.1999.1016>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Haoming Wang¹ · Chengliang Wang¹ · Zhan Chen¹ · Fa Liu¹ · Chunjia Bao² · Xianlong Xu¹ 

✉ Xianlong Xu
xlxu@eec.ecnu.edu.cn

¹ Department of Education Information Technology, Faculty of Education, East China Normal University, Shanghai 200062, China

² Institute of Systems Science, National University of Singapore, Singapore 119615, Singapore